IGLOO Attribution Methods — Formulas

Path-2 investigation reference

2026-09-15

Notation

Symbol	Meaning
$s^0,\ s^1$	wild-type and variant sequence (same length; point substitutions)
$t^0,\ t^1$	tokenized s^0, s^1 — integer token IDs, length $L+2$ (BOS + L residues + EOS)
$E(\cdot)$	ESM-C token-embedding lookup (`client.embed`, an `nn.Embedding`)
e_i^0, e_i^1	$E(t_i^0), E(t_i^1) \in \mathbb{R}^{960}$ — token embedding at position i
$T(\cdot)$	ESM-C transformer stack (`client.transformer`; frozen weights, differentiable)
$x^0,\ x^1$	**final** per-residue embeddings, $x = \text{strip}(T(e))$ — BOS/EOS dropped, shape $(L, 960)$
$\Delta x = x^1 - x^0$	per-residue embedding displacement
$M(\cdot)$	trained predictor head (attention-pool + MLP, or an architecture variant)
F	the composed scalar fitness prediction the method differentiates (defined per-method below)
$\nabla_{x_i} F$	gradient of F w.r.t. the embedding at position i
w	ground-truth position-importance vector (synthetic data generator)
L	protein length (residues)
$\alpha_k = k/\text{steps}$	interpolation fraction at IG step k; w_k = trapezoidal weight ($w_0 = w_{\text{steps}} = \frac{1}{2}$, else 1)

All IG variants below use the same trapezoidal-rule discretization:

$$\int_0^1 \nabla F(\gamma(\alpha))\, d\alpha \;\approx\; \frac{1}{\text{steps}} \sum_{k=0}^{\text{steps}} w_k \, \nabla F(\gamma(\alpha_k))$$

and satisfy the **completeness axiom** $\sum_i a_i = F(\text{target}) - F(\text{baseline})$ exactly (up to discretization error), except where noted.

---

1. Embedding-level methods (current default — `sequence_ig.py`)

These treat ESM-C as a **frozen, precomputed feature extractor**: x^0, x^1 are computed once, offline; only the small predictor head M is differentiated. Here $F(x) = M(x)$.

1.1 IG (signed) — the main IGLOO method

$$a_i \;=\; \Big\langle \int_0^1 \nabla_{x_i} F(x^0 + \alpha \Delta x)\, d\alpha,\;\; \Delta x_i \Big\rangle$$

Product of **model gradient** and **pLM displacement**. Completeness-exact: $\sum_i a_i = F(x^1) - F(x^0)$.

1.2 grad_mag (`gradient`) — factor ablation, gradient only

$$a_i \;=\; \left\| \int_0^1 \nabla_{x_i} F(x^0 + \alpha \Delta x)\, d\alpha \right\|_2$$

Drops the Δx_i weighting — isolates the **model gradient magnitude** alone. Does not satisfy completeness (it's a norm, not a signed decomposition).

1.3 displacement (`input_diff`) — factor ablation, geometry only

$$a_i \;=\; \|\Delta x_i\|_2$$

No model at all — pure pLM embedding geometry. Identical across seeds, noise level, and model quality by construction.

1.4 grad×input (`grad_input`) — classic Shrikumar-style baseline

$$a_i \;=\; \langle \nabla_{x_i} F(x^1),\; x_i^1 \rangle$$

Single-point gradient times input, evaluated **only at** $\alpha = 1$ — no path integral, no baseline.

1.5 ISM (in-silico mutagenesis) — perturbation-based, no gradient

$$a_i \;=\; \frac{1}{19} \sum_{a' \in \mathcal{A},\; a' \neq s_i^0} \big| F(s^0 : i \to a') - F(s^0) \big|$$

where $\mathcal{A}$ is the 20-letter amino-acid alphabet. Runs the whole model $19 \cdot L$ times; no gradient, no embedding-space assumption. A signed variant keeps the mean (not the mean of absolute values).

1.6 Architecture variants — same formula, different M

`IG_premlp`, `channel_attn`, `perres_mean`, `perres_attn`, `crossattn`, and the `onehot` input-representation test all use the **identical** signed-IG formula (§1.1) — only the predictor architecture M (and, for `onehot`, the embedding space itself — 960-dim ESM-C replaced by a 21-dim one-hot amino-acid encoding) changes. See `docs/attribution_methods_and_metrics.md` for what each architecture does structurally.

---

2. Token-level IG (pilot — `scripts/token_level_ig_esmc.py`)

Extends the interpolation **through the entire pLM**, not just the predictor head, while keeping the WT-vs-mutant counterfactual framing. Define the fuller composed function:

$$F(e) \;=\; M\Big(\mathrm{norm}(\mathrm{strip}(T(e)))\Big)$$

so the transformer stack T is now *inside* the differentiated function, not run once offline. Interpolation happens at the **token-embedding layer** e (before T):

$$a_i \;=\; \Big\langle \int_0^1 \nabla_{e_i} F(e^0 + \alpha(e^1 - e^0))\, d\alpha,\;\; (e_i^1 - e_i^0) \Big\rangle, \qquad i = 0, \ldots, L{+}1$$

with the same trapezoidal discretization as above, and the reported per-residue vector drops the BOS/EOS rows: $(a_1, \ldots, a_L)$.

Completeness: $\sum_{i=0}^{L+1} a_i \;=\; F(e^1) - F(e^0)$, verified numerically (`completeness_pred` vs. `completeness_true` in the script's output).

2.1 Why this degenerates for point mutations

E is a pure per-position lookup — e_i depends only on the token *at* position i, with **zero cross-position mixing** (mixing is exactly what T computes, and T has not run yet at this layer). So for any position i where $t_i^0 = t_i^1$ (token unchanged):

$$e_i^1 - e_i^0 \;=\; E(t_i^1) - E(t_i^0) \;=\; \mathbf{0} \qquad \Longrightarrow \qquad a_i = \langle \cdot\,,\, \mathbf{0} \rangle = 0$$

identically, regardless of the gradient term. For a single point substitution, exactly one position can be nonzero; for a double/triple mutant, exactly the 2–3 substituted positions can be nonzero. This is not a bug — it is forced by the same axioms (Sensitivity / Dummy) that make IG a valid attribution method, applied at a layer where, by construction, only the substituted token(s) differ.

2.2 Contrast with the embedding-level method (§1.1)

	Interpolates at	Δ is sparse?	Gradient flows through the pLM's own weights?
Embedding-level IG (§1.1)	final embedding x (post-T)	No — T's self-attention already spread the mutation's effect across positions *before* any gradient is computed	No — T already ran once, offline
Token-level IG (§2)	token embedding e (pre-T)	Yes — exactly the substituted position(s)	Yes — ∇ flows through all of T

There is no formulation that has *both* a non-sparse Δ *and* a gradient computed through the whole pLM for a WT-vs-single-mutant pair: the embedding-level method's spread comes from letting the (untrained, frozen) T act **before** the attribution is computed, not from a gradient. Token-level IG isolates what is formally attributable to the literal input change, stripped of that pre-mixing — informative on its own terms for multi-mutants; degenerate (but correctly so) for single mutants.

---

3. Quick-reference table

Method	Formula	Completeness-exact?	Gradient through the pLM's own weights?
IG (signed)	$\langle \int_0^1 \nabla_{x_i} F \, d\alpha, \, \Delta x_i \rangle$	Yes	No (frozen, precomputed)
grad_mag	$\lVert \int_0^1 \nabla_{x_i} F \, d\alpha \rVert_2$	No	No
displacement	$\lVert \Delta x_i \rVert_2$	n/a (no model)	n/a
grad×input	$\langle \nabla_{x_i} F(x^1), \, x_i^1 \rangle$	No	No
ISM	$\frac{1}{19} \sum_{a'} \lvert F(s^0{:}i{\to}a') - F(s^0) \rvert$	No (not gradient-based)	n/a (perturbation)
Token-level IG	$\langle \int_0^1 \nabla_{e_i} F \, d\alpha, \, \Delta e_i \rangle$	Yes	Yes

---

Generated for the Path-2 investigation, session `d256fa7d`. See `docs/attribution_methods_and_metrics.md` for the evaluation metrics and architecture descriptions, and `scripts/token_level_ig_esmc.py` for the reference implementation of §2.